Using the Franka FR3 and teleoperation setup

Introduction

This guide documents how to reproduce a teleoperation setup combining a Franka FR3 robot arm and a LEAP Hand. The system allows remote control of the robot arm using wrist-mounted motion and hand pose inputs. This guide describes the setup step-by-step, covering hardware and software components, including:

- Low-level control of the robot via libfranka and ROS 2
- MoveIt Servo for real-time Cartesian control
- A wrist-worn teleoperation device using a camera, a HTC Vive Ultimate tracker and a haptic feedback system
- Hand pose tracking and and retargeting to the LEAP Hand using DexRetargeting
- Camera integration for visual feedback
- Tactile sensor integration for deformation and force feedback
- Integration of all components for full system execution

GitHub repository

This GitHub repository contains the working setup for operating the Franka FR3 and the LEAP Hand. It extends the official franka_ros2 repository and includes:

- The default contents of franka_ros2 (see 2.1 What is franka_ros2), adapted to the Franka FR3
- MoveIt Servo, modified for our usage (see 3.1 What is MoveIt Servo?)
- A custom Moveit launch file (moveit_pose_cam.launch.py) (see 3.3.1 Modify the launch file)
- servo_keyboard_input (see 3.4 Using the Joystick Input or Keyboard Input (For Testing))
- Description files of the LEAP Hand end-effector (see 4.3 LEAP Hand Description and Integration with ROS 2)
- The LEAP Hand API (ros2_module)
- A wrapper for DexRetargeting (dex_retargeting package), including a modified optimizer.py (see 4. Integrating the LEAP Hand)
- A custom fr3_leap_teleop package (containing teleop_vive_leap_ros2.py) to publish hand poses (see 6.2 Integrating LEAP Hand retargeting and Vive tracker pose publishing) and wrist poses (see 5. Streaming Wrist Pose with HTC Vive Tracker and DexCap)
- The camera calibration package easy_handeye2, including a custom Aruco marker detector (see 7.Visual feedback)
- A ROS2 wrapper for the tactile sensor software 9DTact (see 8. Tactile feedback)
- A custom fr3_leap_recorder package (containing fr3_leap_recorder.py) (see 9. Data collection)

Not included in the repository:

- The script to publish HTC Vive tracker data from the Windows PC

Installation

Following the steps in this guide should allow any user to replicate the setup contained in this repository by installing the different components from their respective source, and adapt it to a different setup.

The repository can also be installed directly:

```
git clone https://github.com/paulm-08/franka_ros2_fr3_teleop.git
rosdep install --from-paths src --ignore-src -r -y
colcon build
source install/setup.bash
```

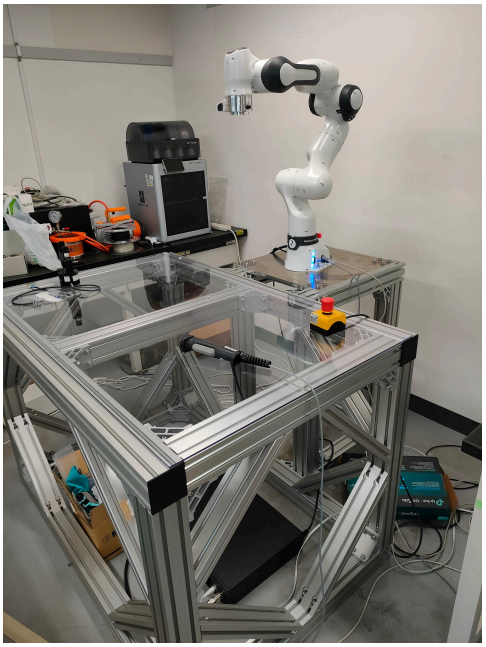
To set up the Windows PC, follow the instructions in 5. Streaming Wrist Pose with HTC Vive Tracker and DexCap.
To script to upload to the haptic feedback device is

Launch the teleoperation system:

```
ros2 launch franka_fr3_moveit_config moveit_pose_cam.launch.py
```

1. Setting Up the Franka FR3 Robot

1.1 Hardware and Network Configuration



The Franka FR3 system includes:

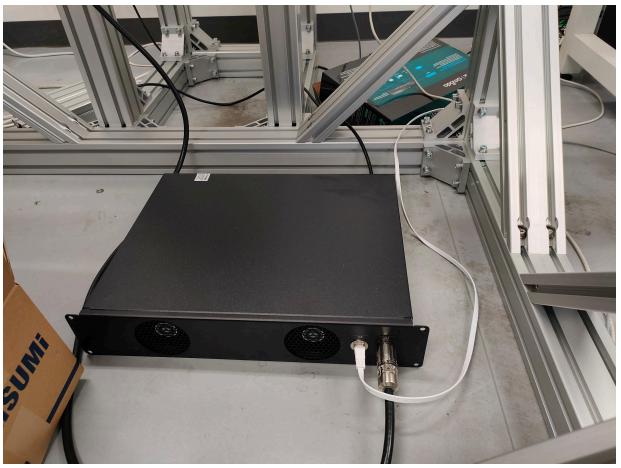
- Robot arm: exposes an Ethernet port at the base
- Control box: separate unit with its own Ethernet port and power switch

Setup options:

- For high-level control (Franka Desk): Connect your PC directly to the robot arm via Ethernet.
- For low-level control (libfranka/FCI): Connect your PC to the control box via Ethernet.

Note: If direct connection doesn't work, try using an Ethernet switch. However, this may cause dropped packets and result in a *communication_constraints_violation* exception in libfranka.

Turn on the robot using the switch on the back of the control box.



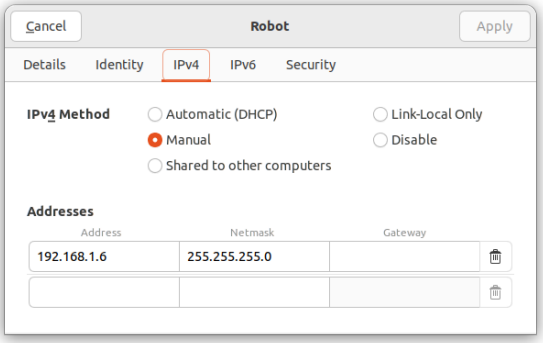
1.2 Accessing Franka Desk

Franka Desk provides a web interface to configure and control the robot.
Video Tutorials:

Franka Emika Tutorial 1 - Desk
Franka Emika Tutorial 2 - Pilot

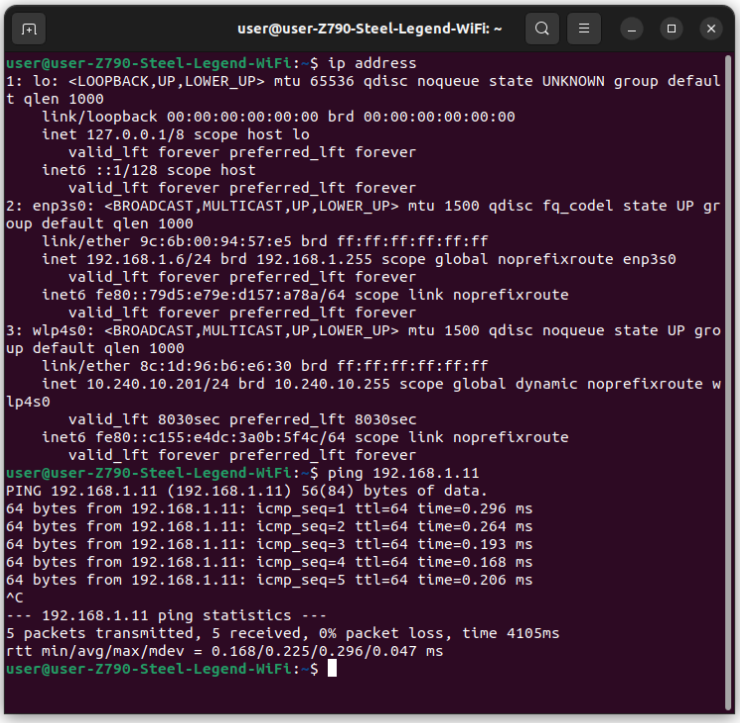
Connecting via browser:

- When connected to the robot arm, visit: http://192.168.0.1 or https://robot.franka.de/desk/
- When connected to the control box, first set your PC's IP manually (e.g., 192.168.1.6, netmask 255.255.255.0), then visit: http://192.168.1.11

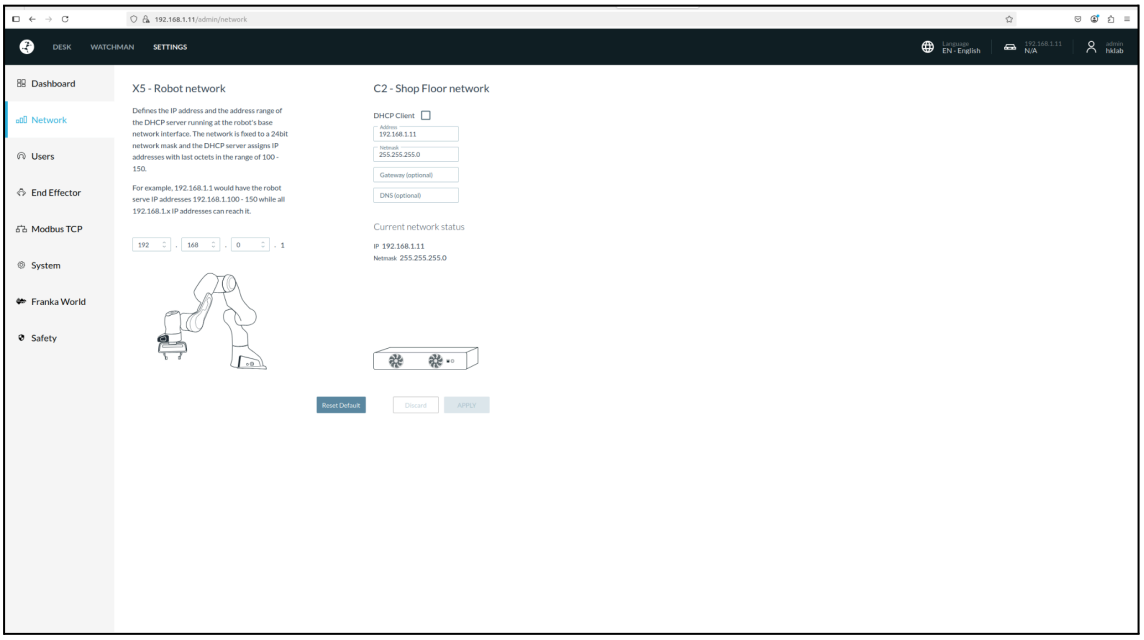


In case of trouble:

```
ip a              # Check IPs assigned to your interfaces
ping 192.168.0.1  # Ping robot arm
ping 192.168.1.11 # Ping control box
```



If only one side is reachable, access Desk from the robot arm first, then inspect or change the control box's network settings (under “Shop Floor Network”).



1.3 Desk Features and Usage

Key features:

- Unlock the brakes
- Switch between:
    - Programming mode: Create and test high-level task sequences using the External Enabling Device.

You can also drag the robot holding by the Guiding Button and half-pressing the Enabling Button on the grip.

- Execution mode: Run tasks and activate FCI (for libfranka control)

Important: Only one user can control the robot at a time. If the previous user hasn't released it, press the round button on the grip to force control transfer.

2. Installing and Using franka_ros2

Video tutorial:
Controlling Franka Research 3 via FCI

2.1 What is franka_ros2

franka_ros2 provides the official ROS 2 integration for the Franka Research 3 robot. It acts as a bridge between libfranka (the C++ API for real-time control of the robot) and ROS 2's ecosystem for motion planning, controller management, and simulation.

This package includes:

- Launch files for running the robot or simulation in ROS 2
- Controller setups for both **position** and **Cartesian** control
- A full URDF of the robot (description + visual/kinematic data)
- Integration with **MoveIt 2** for planning and motion execution
- Example code for implementing your own real-time controllers

We use it to run real-time control code and send low-level commands to the robot. A real-time kernel is recommended because commands are sent to the robot at 1 kHz but it is not mandatory. While libfranka handles real-time communication at 1 kHz, franka_ros2 abstracts much of that complexity through standard ROS 2 topics, services, and parameters.

2.2 libfranka Overview

libfranka is a C++ library developed by Franka Emika for low-level control of the robot. It is designed for real-time applications and provides:

- Access to robot state at 1 kHz
- APIs to send torque, joint position, and Cartesian pose commands
- Safety mechanisms to enforce torque and motion limits

Important: libfranka requires a real-time Linux kernel to work at its intended frequency (1 kHz). Without it, its example programs won't execute properly. However, franka_ros2 can be used without a real-time kernel thanks to ROS 2's built-in scheduling and buffering.

2.3 Installing ROS 2 Humble

Before installing franka_ros2, follow the official ROS 2 Humble installation guide for Ubuntu 22. Install the desktop version.

Make sure your Linux PC is running Ubuntu 22, as franka_ros2 targets ROS 2 Humble.

2.4 Cloning and Building franka_ros2

The franka_ros2 GitHub page provides a ROS 2 Integration for Franka Robotics Research Robots which includes libfranka. libfranka can also be installed separately from the libfranka GitHub page.

Follow the installation instructions of franka_ros2. Always source the workspace in every terminal.

2.5 What's Included in franka_ros2

Once built, your workspace will contain:

- **franka_bringup/**
  Launch files to start the robot, real or simulated.
- **franka_example_controllers/**
  Sample ROS 2 controllers that use libfranka interfaces. These include Cartesian impedance controllers, joint controllers, etc.
- **franka_description/**
  URDF, mesh, and kinematic configuration files for the FR3 arm.
- **franka_fr3_moveit_config/**
  MoveIt 2 configuration package with planners, controller settings, and robot SRDF (semantic description). Includes launch files for visualization in RViz.

2.6 Testing the Setup with MoveIt and RViz

To verify that your installation and robot connection are working, try launching the included MoveIt visualization with real or fake hardware:

- Real hardware:
  ```
  ros2 launch franka_fr3_moveit_config moveit.launch.py robot_ip:=192.168.1.11
  ```
- Fake hardware:
  ```
  ros2 launch franka_fr3_moveit_config moveit.launch.py robot_ip:=192.168.1.11 use_fake_hardware:=true
  ```

This will:

- Launch MoveIt's planning pipeline
- Load the FR3 arm's URDF/SRDF
- Start controllers from franka_ros2
- Open RViz, where you can visualize the robot and send planning requests

3. Using MoveIt Servo for Real-Time Cartesian Control

[Realtime Arm Servoing tutorial for MoveIt 2 Humble]
[Github page for MoveIt Servo]

3.1 What is MoveIt Servo?

MoveIt Servo allows real-time control of robot end-effectors by converting small pose or velocity updates into joint commands at high frequency. It's ideal for teleoperation and interactive control, and works with standard ROS 2 topics like /servo_node/delta_twist_cmds or /target_pose.

Two key operation modes:

- Twist-based control:
  - Control by Cartesian velocity
  - The twist commands are published to the ROS2 topic /delta_twist_cmds
- Pose tracking:
  - Control by absolute Cartesian poses
  - The pose commands are published to the ROS2 topic /target_pose
  - The cartesian poses are internally converts to velocity commands using a PID controller. The pose tracking node calls the twist-based servo node and publishes the computed velocities to the ROS2 topic /delta_twist_cmds

Which one to use?

- Use twist mode if your input is based on velocities or joystick/keyboard deltas.
- Use pose tracking if you're streaming absolute poses (e.g., from a VR tracker).

MoveIt Servo is not part of the franka_ros2 packages included by default, so we need to add it to the workspace.

3.2 Installing MoveIt Servo

Install and build MoveIt Servo in the same workspace as franka_ros2.

3.3 Integrating MoveIt Servo with franka_ros2

To enable MoveIt Servo in your Franka setup, you need to modify the MoveIt launch and config files.

3.3.1 Modify the launch file:

Modify the MoveIt launch file to use MoveIt Servo instead of the move_group module:
franka_fr3_moveit_config/launch/moveit.launch.py
For example, save it as a moveit_pose.launch.py.

Add the servo node or the pose tracking node and load the configuration files.

3.3.2 Create or edit config files:

- For the base servo module:
    - We only need one config file like /moveit_servo/config/panda_simulated_config.yaml
    - Edit the file to match the FR3 nomenclature. In particular, change:
        - ‘command_in_type’ to ‘unitless’ if using a joystick/keyboard input and ‘speed_units’ otherwise
        - ‘move_group_name’ to ‘fr3_arm’
        - ‘planning_frame’ to ‘fr3_link0’ or ‘base’
        - ‘ee_frame_name’ to ‘fr3_link8’
        - ‘robot_link_command_frame’ to ‘fr3_link0’ or ‘base’
        - ‘cartesian_command_in_topic’ to ‘/servo_node/delta_twist_cmds’
        - ‘joint_command_in_topic’ to ‘/servo_node/delta_joint_cmds’
        - ‘joint_topic’ to ‘/franka_robot_state_broadcaster/joint_states’
        - ‘command_out_topic’ to ‘/fr3_arm_controller/joint_trajectory’
- For the pose tracking module:
    - We need a config file similar to the previous one.
      /moveit_servo/config/panda_simulated_config_pose_tracking.yaml
      The only difference with the config file for the basic servo node:
        - Change ‘robot_link_command_frame’ to ‘fr3_link8’
        - Change ‘command_in_type’ to ‘speed_units’
    - We need a pose tracking specific config file.
      /moveit_servo/config/pose_tracking_settings.yaml
      Edit the PID gains here.

3.3.4 Modify the pose tracking

The pose tracking node calls an executable called servo_pose_tracking. This executable is built from the source file pose_tracking_demo.cpp.
This demo file enables pose tracking but publishes its own dummy trajectory to /target_pose. Comment out the lines publishing the fake data and rebuild the package.

3.3.5 Tune the PID controller

Tune the PID gains to improve response time but avoid instability and jittery motion.
Suggested gains:

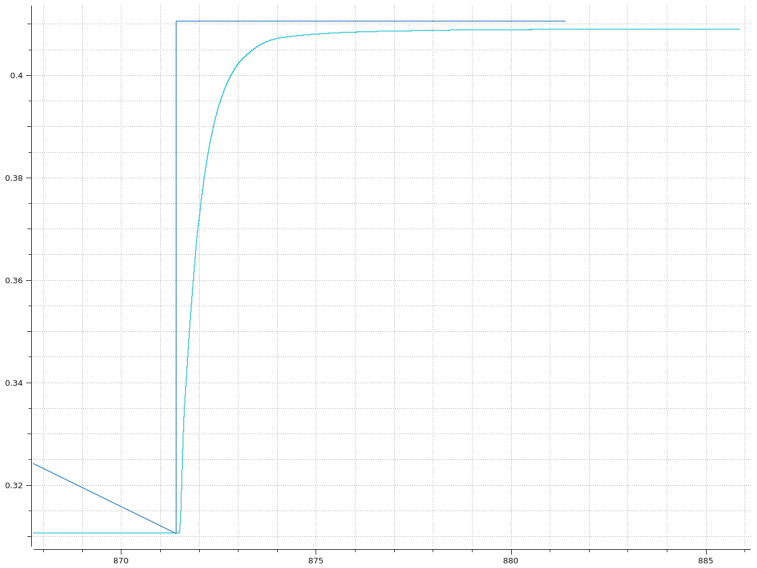
- 12.0 for the proportional gains (linear and angular)
- 0.3 for the derivative gains (linear and angular)

Example of a standard test with step command:
The pose tracking demo is modified to send a single pose, as a step command amplitude: 0.1m in x coordinate.

```
# Maximum value of error integral
for all PID controllers
windup_limit: 0.05

# PID gains
x_proportional_gain: 4.0
y_proportional_gain: 4.0
z_proportional_gain: 4.0
x_integral_gain: 0.0
y_integral_gain: 0.0
z_integral_gain: 0.0
x_derivative_gain: 0.5
y_derivative_gain: 0.5
z_derivative_gain: 0.5

angular_proportional_gain: 4.0
angular_integral_gain: 0.0
angular_derivative_gain: 0.5
```



Observations:

- Amplitude
    - command 0.100
    - response 0.098
    - error 0.002
    - error 2%
    - linear tolerance 0.01 ~ 10% amplitude
- Rise time
    - 95% of final position = 0.403
    - rise time to 95% = 873.1-871.5 = 1.6s
- Overshoot
    - None
- Oscillations
    - None

⇒ In this case, the controller is too conservative (stable but laggy) so proportional gains can be increased.

3.3.3 Launch

Running the launch file activates the servo node. When using the servo node, start servoing by sending a trigger signal in another terminal:

```
ros2 service call /servo_node/start_servo std_srvs/srv/Trigger {}
```

3.4 Using the Joystick Input or Keyboard Input (For Testing)

MoveIt Servo provides a package to teleoperate a robot with a controller's joystick or simply with a keyboard. Follow the Realtime Arm Servoing tutorial for MoveIt 2 Humble.

Joystick (not tested)

Install the joy package and add a joy node in the launch file. The controller should be detected automatically.

Keyboard

1. Install the servo_keyboard_input executable. It comes with MoveIt2 but isn't included in the default franka_ros2 workspace, so we can just add it to our packages.
2. In the the source file servo_keyboard_input.cpp, modify the constants to match your setup before rebuilding:

```
// Some constants used in the Servo Teleop demo

const std::string TWIST_TOPIC = "/servo_node/delta_twist_cmds";

const std::string JOINT_TOPIC = "/servo_node/delta_joint_cmds";

const size_t ROS_QUEUE_SIZE = 10;

const std::string EEF_FRAME_ID = "fr3_link8";

const std::string BASE_FRAME_ID = "base";
```

3. Launch your MoveIt configuration with Servo.
4. In a new terminal, source your workspace and run the servo_keyboard_input executable:

```
ros2 run servo_keyboard_input servo_keyboard_input
```

This publishes to /servo_node/delta_twist_cmds, which should move the robot in real time. Check with rqt_graph or ros2 node info /keyboard_pose_publisher.

Note: The joystick and keyboard control only works with the twist mode.

4. Integrating the LEAP Hand

4.1 What is the LEAP Hand?

Paper
The LEAP Hand is a custom-designed robotic hand used for dexterous manipulation. In this setup, it is mounted on the Franka FR3 robot arm and controlled using motor commands derived from vision-based hand tracking (via DexRetargeting).

It provides:

- 5 degrees of actuation (DoA)
- USB-based communication
- Positional or velocity-based joint control modes

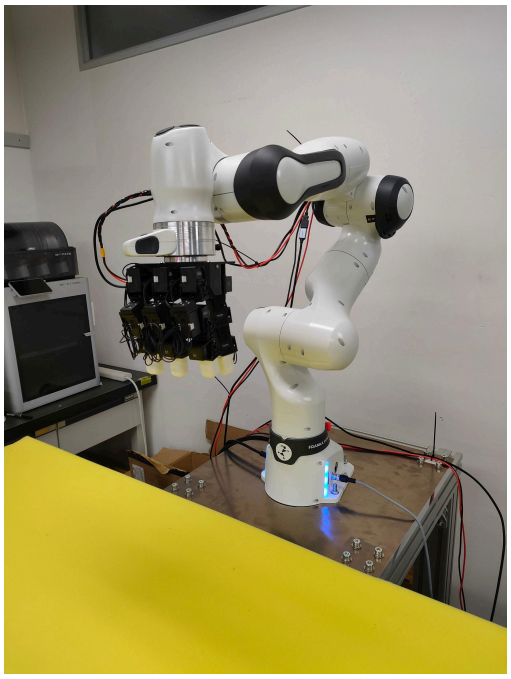
4.2 Hardware Setup

Building tutorial
Additional notes from Lai

You need to connect:

- Power supply to the LEAP Hand (separate DC input)
- USB cable from the LEAP Hand to your Linux PC

After building the hand, it can be mounted physically on the Franka FR3 arm as an end-effector. Since this setup doesn't use the end-effector outlet of the Franka FR3 arm for power and communication, a USB extender and long power cables are run along the arm.



4.3 LEAP Hand Description and Integration with ROS 2

To simulate or visualize the LEAP Hand in ROS 2 (e.g. in RViz or MoveIt), you'll need its URDF/XACRO files and include them in your robot's description.

If you haven't already:

1. Obtain the URDF/XACRO files for the LEAP Hand from the LEAP Hand CAD page.
2. Add them to your ROS 2 workspace (e.g., franka_ros2_ws/src/leap_hand_description).
3. Create a new package in src called "leap_description", containing:
    a. meshes: folder containing the .stl files
    b. parts: folder containing the .part files
    c. urdf: folder containing the URDF file
    d. CMakeLists.txt
    e. package.xml
4. Create a new folder "leap_hand" in src/franka_description/end_effectors
5. Create inside this folder:
    a. leap_hand.urdf.xacro: the URDF file of the LEAP hand converted to xacro with a wrapper, with visuals linked to the respective mesh files (corrected to the new file configuration) and collisions deleted (easy way out of collision issues but causes warnings)
    b. leap_hand_arguments.xacro: defined some variables, e.g. the orientation and origin of the end-effector to the parent link fr3_link8. Change the values to position the LEAP hand correctly relative to the arm.
6. Add a condition block in src/franka_description/robots/common/franka_robot.xacro: if ee_id=='leap_hand' → load leap_hand.urdf.xacro
7. Change the launch file to have hand=True and ee_id='leap_hand'
8. Then, rebuild and re-source your workspace:

```
cd ~/franka_ros2_ws
colcon build --cmake-args -DCMAKE_BUILD_TYPE=Release
source install/setup.bash
```

You should now be able to visualize the hand attached to the FR3 in RViz when launching moveit.launch.py.

4.4 Controlling the LEAP Hand with DexRetargeting

Hand control is provided through DexRetargeting, a framework that extracts human hand poses from camera images using MediaPipe and translates them into LEAP Hand joint commands.

We use:

- A USB camera mounted on the wrist (facing the hand)
- MediaPipe to detect the hand skeleton
- Custom Python code to convert the detected poses into motor commands
- The Dynamixel client to write the command poses to the motors

4.5 Installing and Running DexRetargeting

1. Clone the project:
   ```
   pip install dex_retargeting
   git clone git clone https://github.com/dexsuite/dex-retargeting
   cd dex-retargeting
   pip install -e ".[example]"
   ```
2. Source the Python environment:
   ```
   source venv-retarget/bin/activate
   ```
3. Connect the hand and camera:
   - Plug in the LEAP Hand's power and USB cable
   - Connect your USB camera to the Linux PC
   - Mount the camera facing your own hand
4. Run the controller:
   ```
   python example/vector_retargeting/denso_control.py --robot-name leap
   --retargeting-type position --hand-type right
   ```

This will:

- Capture video from the camera
- Run hand pose estimation using MediaPipe
- Send joint position commands to the LEAP Hand

4.6 Tuning DexPilot for our specific task

By default, DexRetargeting uses one of 3 optimizers: position, vector and DexPilot. From the keypoints extracted by the hand detector, each optimizer defines and optimizes a loss function to match the LEAP Hand pose to the target pose.

4.6.1 What is DexPilot?

DexPilot is an optimizer that enables stable grasping by forcing the fingers in a closed position once the target fingers are close enough.

DexPilot uses vectors between each of the keypoints at the fingertips and on the wrist and minimizes the distance between the target vectors (calculated from the hand detectors) and the robot vectors. When a vector between the thumb and another finger (S1 vector) is small enough, it is recognized as a grasp and the vector is projected to a fixed distance (close to 0, to model a contact). The vectors between primary fingers (S2 vectors) are projected when the S1 vectors of both fingers are already projected. When vectors are projected, their

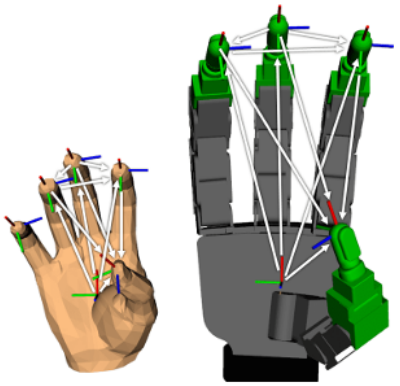


Fig. 8. Task space vectors between fingertips and palm for both the human hand model and Allegro hand used for retargeting optimization.

TABLE I. Description of vector sets used in kinematic retargeting.

Set	Description
$\mathcal{S}_1$	Vectors that originate from a primary finger (index, middle, ring) and point to the thumb.
$\mathcal{S}_2$	Vectors between two primary fingers when both primary fingers have associated vectors $\in \mathcal{S}_1$, *e.g.*, both primary fingers are being projected with the thumb.

assigned weight in the loss function is increased as well. This keeps the fingers stable during grasping.

4.6.2 Enforcing a pinch grasp

When the LEAP Hand is controlled by DexPilot, it grasps objects by making contact at the fingertips. However, in the case of DLO grasping, this behavior makes it difficult to grab the object as the contact surface is rounded and very small. Additionally, when implementing tactile sensors(e.g. 9DTact), we need the contact point to be on the side of the fingers. This type of contact is akin to the way humans naturally grasp DLOs, forming a “pinch” grasp with the soft side surfaces of the fingers parallel.

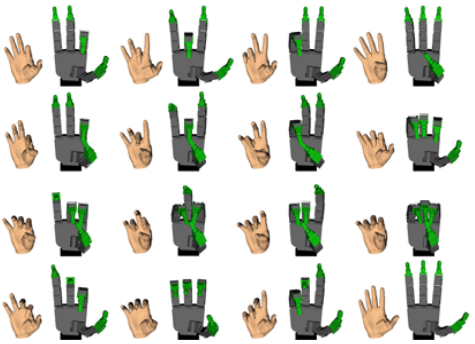
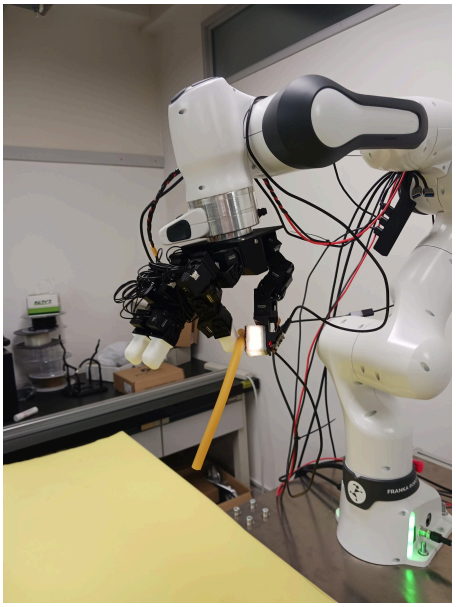


Fig. 9. Canonical kinematic retargeting results between the human hand model and the Allegro hand model.



These observations justify the need for a tweak in the logic of the DexPilot optimizer to facilitate DLO manipulation. We make the following modifications:

1. Create new virtual auxiliary links in the URDF, tied to the fingertips (with an offset on x). DexPilot already introduces virtual links to define the position of the fingertip point to be controlled.
2. Using the new virtual links, define one vector per fingertip (index and thumb), normal to the contact surface of the fingertips.
3. In DexPilot, compute the cosine similarity of the vectors to calculate a loss on parallelism of the fingers. Adding this new loss to the optimizer with weights enforces a “pinch” grasp. When the finger vectors are not projected, set the parallelism loss to 0.
4. Keep the rest of the logic intact and make sure the computed gradient is correct. The LEAP Hand keeps its original behavior for the middle finger and the ring finger, and when the thumb and index finger are not projected.

Note: Enforcing parallelism in 2 directions instead of just the normal vectors can prevent the surfaces from pivoting and keep them in a position similar to a 1DoF gripper.

4.6.3 Allow intermediate grasping distances

Due to the DexPilot projection method, there is an issue of the fingers “snapping” to the closed grasp pose directly from an open grasp. This is a problem for DLO grasping where precise positioning of the fingers is needed to place the object in between the fingertips. A human demonstrator operating the robot needs visual feedback that the fingertips are properly aligned with the object before actually grasping. To provide this possibility, we extend the original logic of the DexPilot optimizer:

1. Increase the projection distance and escape distance: The projection distance is the threshold below which a vector (e.g., from thumb to index) is considered close

enough to be “projected,” i.e., snapped to a grasping configuration. The escape distance is the threshold beyond which projection is removed. Increasing both thresholds allows projection to activate earlier and remain active longer, giving a broader window for stable pre-grasp alignment.

2. Define an adaptive projection distance function: Instead of using a fixed projected length (typically close to zero), we dynamically compute the projected distance based on the actual distance between the target fingers. This allows intermediate grasp distances, enabling fine positioning of the object before a tight grasp is formed. The function may be defined as:

```
def adaptive_projected_distance(d, d0=0.03, min_eta=1e-4,
d_max=escape_dist, max_eta=escape_dist):
    if d <= d0:
        return min_eta
    elif d >= d_max:
        return max_eta
    else:
        t = (d - d0) / (d_max - d0)
        return min_eta + t * (max_eta - min_eta)
```

   This function linearly interpolates between a near-zero projection length (same as original) for close contact and a projected length equal to the escape distance to avoid sudden snapping into the grasp.

3. Define adaptive weights based on target vector distance:
   To further smooth the grasping transition and allow for fine motion near contact, we define a distance-dependent weighting function. The weight increases as fingers come closer together, which gradually tightens the retargeting constraint and increases stability during the approach. A sample function may be:

```
def adaptive_weight(d, min_d=0.03, max_d=escape_dist, w_min=1.0,
w_max=200.0):
    d_clipped = np.clip(d, min_d, max_d)
    return w_min + (max_d - d_clipped) / (max_d - min_d) *
(w_max - w_min)
```

   This logic makes the optimizer more responsive when the fingers are close (e.g., about to grasp) and more permissive when fingers are far (e.g., during reach or repositioning).

4. Apply projection and weights jointly:
   The projection status is still binary (based on the vector being within the projection and escape distance thresholds) but the actual projection target and its influence on the optimizer are made smooth through the adaptive functions. When projection is active, we use the computed adaptive projection distance and apply the corresponding adaptive weight; otherwise, we fall back to the original target vector and default weight.

4.7 Integration with ROS 2 (not implemented)

If you want to control the LEAP Hand alongside the robot in a coordinated way (e.g., publish both arm and hand commands in sync), you can use the LEAP Hand API. This integration is optional but helpful if you're doing synchronized manipulation or logging.

In this setup, we control the LEAP Hand using the Dynamixel API directly (in the python script handling wrist pose publishing and retargeting hand poses). We use the LEAP Hand API to read the joint positions in the data recording script.

5. Streaming Wrist Pose with HTC Vive Tracker and DexCap

To teleoperate the Franka FR3 arm, we track the operator's wrist using an HTC Vive Ultimate Tracker. This section explains how to set up the trackers using SteamVR and Vive Hub, stream their poses from a Windows PC over UDP, and receive and integrate the poses on a Linux PC.
We follow a pipeline inspired by the DexCap project.

5.1 Hardware Requirements

- HTC Vive Ultimate Tracker
- Vive USB dongle (paired with tracker)
- Windows PC with Steam and Vive Hub
- Linux PC connected to the FR3 control and LEAP Hand
- Local area network (both PCs on the same subnet)

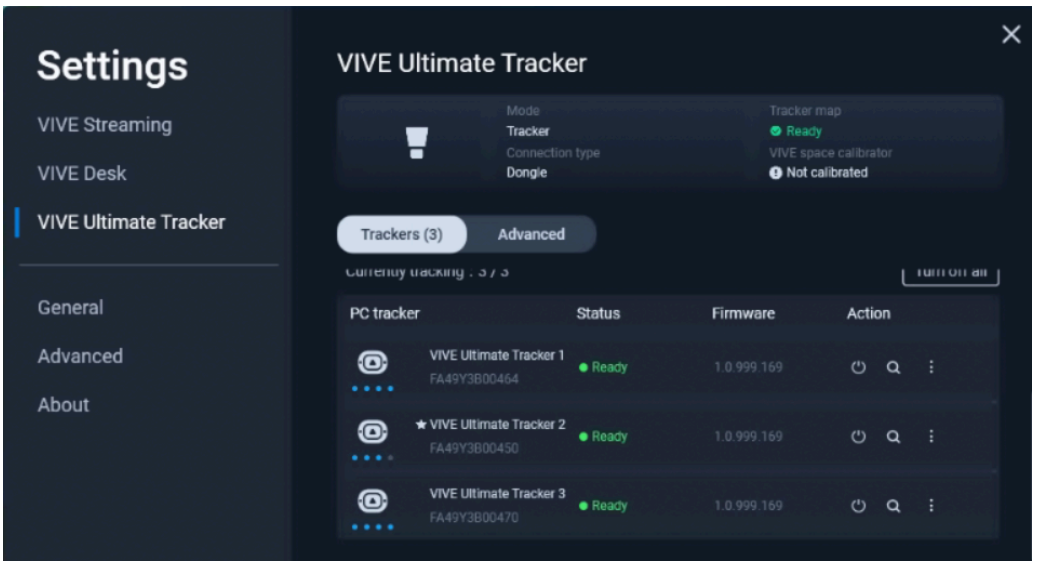
5.2 Software Setup on the Windows PC (first time)

Follow the DexCap Hardware Tutorial (VIVE tracker setup section).
The VIVE trackers require a Windows PC with Steam (a Steam account is needed). The data from the trackers are sent to the Windows PC and can then be streamed to the Linux workstation.

5.3 Publish data from the trackers

If the trackers are already setup and the room is mapped:

1. Connect the USB dongle to the Windows PC
2. Launch Steam VR and VIVE Hub
3. In VIVE Hub, go to Settings > Ultimate trackers
4. Turn on the trackers and place them at the center of the room (if they show "Lost tracking", they're not placed correctly compared to the room mapping)
5. When they show Ready, we can run code:



6. (Optional) Launch an Anaconda terminal and run a test to see if the trackers data is received:
   ```
   conda activate dexcap
   cd DexCap/STEP1_collect_data_202408updates
   python vive_test.py
   ```
7. To publish the data from the trackers, check the UDP_IP and UDP_PORT in the `vive_publisher_udp.py` file (should be 10.240.10.201, the IP of the Linux workstation—check it by running `ip a` on the Linux workstation) and run it:
   ```
   conda activate dexcap
   cd DexCap/STEP1_collect_data_202408updates
   python vive_publisher_udp.py
   ```

5.5 Receiving Data on the Linux PC

On the Linux workstation, you can use a test script or integrate directly with your ROS 2 control code.

Option A: Visualization/Test

```
/bin/python3
/home/user/DexCap/STEP1_collect_data_202408updates/vive_udp_toy_receiver_vis.py
```

This script listens to the UDP stream and visualizes the positions of the trackers in 3D.

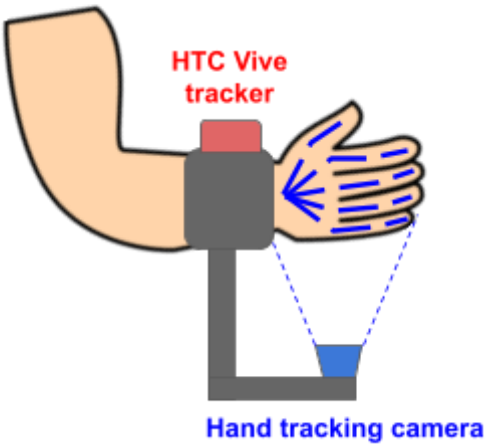
Option B: ROS 2 Integration

To teleoperate the Franka FR3, we use a custom script to listen to the UDP stream, process the data and publish the computed command to the corresponding ROS2 topic (/target_pose if we're using MoveIt Servo in pose tracking mode).
Refer to 6.2 Integrating LEAP Hand retargeting and Vive tracker pose publishing.

6. Wrist-Mounted Teleoperation Device

6.1 Overview of the design

To teleoperate the arm, we physically mount the Vive tracker on a wrist-worn device that also holds the USB camera used for hand pose estimation. This setup allows a human operator to control both the Franka FR3 arm and the LEAP Hand.

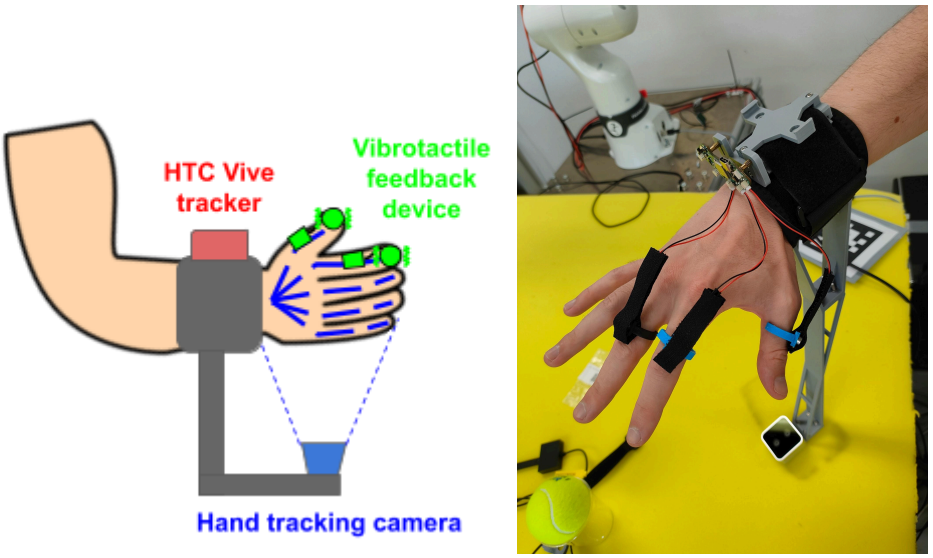


Design of the wrist attachment:

- Mount the Vive tracker on top of the wristband.
- Mount the USB camera on the palm side, pointing at the hand.

Video of a test:
https://youtu.be/BvM6Gs7u3hs

Improvement: Adding a vibrotactile actuator to provide feedback of the robot's tactile sensors. See Use case: Haptic feedback.



6.2 Integrating LEAP Hand retargeting and Vive tracker pose publishing

In a custom Python script, we integrate the following functions in an asynchronous loop:

- Vive tracker pose publishing
  - Receive the UDP stream
  - Filter the signal to avoid jittery commands (EMA filtering)
  - Convert the pose data into a geometry_msgs/msg/PoseStamped
  - Publish it to /target_pose

  →Controls the Franka FR3
- LEAP Hand retargeting
  - Process the camera stream
  - Run the DexRetargeting controller
  - Write the obtained poses to the motors

  → Controls the LEAP Hand
- Visualizations of the camera stream and the tracker poses

The custom script teleop_vive_leap_ros2.py is available in the GitHub repository, in the corresponding package: fr3_leap_teleop
Make sure to have DexRetargeting installed and specify the path in the code to resolve the import.

Notably, the Vive tracker coordinate space and the robot cartesian space are different. To solve this issue:

1. We convert the axes of the Vive tracker coordinate system to that of the robot.
   ⇒ $x_R = -z_V ; y_R = -x_V ; z_R = y_V$

```
                position = np.array([
                    -pose["position"]["z"],  # ROS X = -VIVE Z (forward/backward)
                    -pose["position"]["x"],  # ROS Y = -VIVE X (left/right)
                    pose["position"]["y"]    # ROS Z = VIVE Y (up/down)
])
```

```
# Convert orientation (quaternion) from VIVE to ROS
# VIVE quaternion: [w, x, y, z]
q_vive = [
    -pose["orientation"]["z"],
    -pose["orientation"]["x"],
    pose["orientation"]["y"],
    pose["orientation"]["w"]
]
```

2. The MoveIt launch file sends a trigger signal via a ROS2 topic on launch. Upon receiving this trigger signal, the Python publisher sets the origin of the published target poses to the position of the robot at trigger time. This avoids unresolvable gaps between the robot pose and the desired pose by matching the initial target position with the initial robot position.

6.3 Launching the full system

Step-by-step:

1. Turn on the robot and access Franka Desk to activate FCI.
2. On the Windows PC:
    a. Launch SteamVR and Vive Hub
    b. Turn on the tracker and start publishing with vive_publisher_udp.py
3. On the Linux PC:
    a. Plug in the LEAP Hand (power + USB)
    b. Plug in the USB camera (USB)
    c. Attach the wrist device
    d. Start the ROS 2 system:
    ```
    source ~/franka_ros2_ws/install/setup.bash
    ros2 launch franka_fr3_moveit_config moveit_pose_cam.launch.py
    robot_ip:=192.168.1.11
    ```
4. At this point, you can control the Franka FR3 + LEAP Hand with your own hand motion and wrist pose in real time. Mind the small delay before the pose tracking starts.

7. Visual feedback

7.1 Overview

Visual feedback provides essential information for perception-driven robot manipulation, enabling the robot to:

- Locate objects, estimate their poses or deformation states,
- Track changes in the workspace
- and adapt motion based on real-world observations

In this setup, we use a fixed RGB-D camera (eye-on-base configuration) to estimate the pose of a reference ArUco marker and reconstruct part of the scene. The camera is calibrated with respect to the robot base frame, allowing us to localize targets in robot coordinates.

2 conventions; Eye-on-base vs Eye-in-hand:

- Eye-on-base: Camera mounted on a fixed frame (e.g., base or workspace).
  Eye-in-hand: Camera mounted on the robot wrist.

7.2 Setup

7.2.1 Hardware

- Camera: Intel RealSense D435i (or D405)
- Mount: Rigidly attached to a fixed structure, aligned to face the workspace.
- Marker: ArUco marker placed on the end-effector during calibration and later on scene objects for localization.

7.2.2 Software

The visual feedback pipeline includes:

1. Camera Drivers

If not done already (e.g. for the teleoperation device), install the realsense2_camera ROS 2 package to access RGB and depth streams.

Launch the camera node:

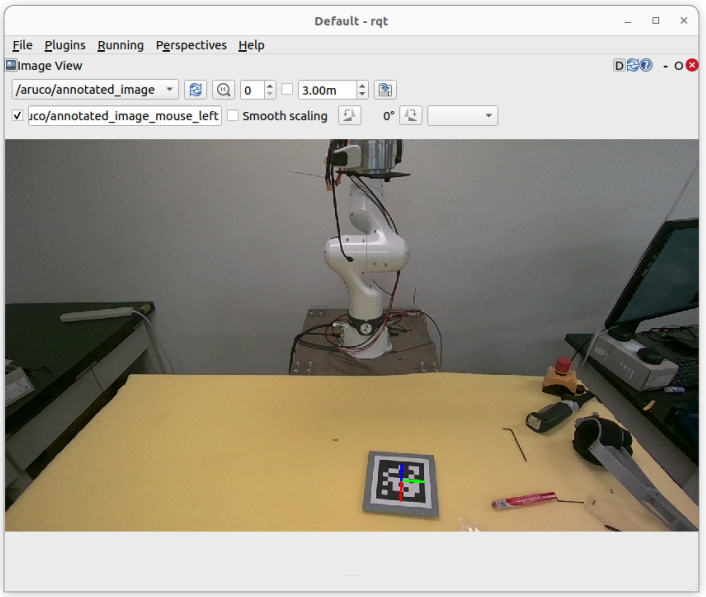
```
ros2 launch realsense2_camera rs_launch.py
```

This publishes image and depth data as well as camera intrinsic parameters to the respective topics:
/camera/camera/color/camera_info
/camera/camera/color/image_raw,
/camera/camera/depth/image_raw

2. Marker Pose Estimation

A custom OpenCV-based ROS 2 node, added to the easy_handeye2 package (see next):

- Subscribes to the color image and camera info topics.
- Detects an ArUco marker.
- Publishes the pose of the marker relative to the camera as a TF transform (e.g., camera_link → aruco_marker).
- Optionally, publishes the axes on the detected marker to the /aruco/annotated_image to validate the tracking



This transform is used either:

- During calibration (marker on end-effector)
- During usage (marker on a fixed target)
- In data processing

Start the publisher:

```
ros2 run easy_handeye2 aruco_tf_publisher --ros-args \
  -p marker_id:=0 \
  -p marker_size:=0.10 \
  -p camera_frame:=camera_link \
  -p tracking_marker_frame:=aruco_marker_frame
```

3. Eye-to-base Calibration Publisher

Install the [easy_handeye2 package].
This package contains a calibration script that saves the calibration to a .yaml file, and a publisher script that fetches
After calibration (see below), the easy_handeye2 publisher script publishes the fixed transform from camera_link to base. This transform must be present in /tf during operation.

Use the calibration in runtime to publish the static transform to /tf:
Run:

```
ros2 launch easy_handeye2 publish.launch.py name:=my_handeye_calib
```

We can add our custom ArUco marker publisher script in the package (e.g. in easy_handeye2/easy_handeye2/easy_handeye2/aruco_tf_publisher.py).

4. Launch File Integration

We include the camera, marker tracker, and calibration transform publisher in the ROS 2 launch system:

```
    # Launch the realsense camera node to publish frames to /tf
    realsense_node = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            PathJoinSubstitution([FindPackageShare("realsense2_camera"), "launch",
"rs_launch.py"])
        ),
    )

    # Launch the Aruco marker pose publisher
    aruco_tf_publisher = Node(
        package='easy_handeye2',
        executable='aruco_tf_publisher',
        name='aruco_tf_publisher',
        output='screen',
        parameters=[
            {'marker_id': 0,  # Change this to your marker ID
             'marker_size': 0.10,  # Size of the marker in meters
             'camera_frame': 'camera_link',  # Frame of the camera
             'tracking_marker_frame': 'aruco_marker_frame',  # Frame for the marker
            #  'camera_matrix': [600, 0, 320, 0, 600, 240, 0, 0, 1],  # Example camera
matrix
            #  'dist_coeffs': [0.1, -0.25, 0, 0]  # Example distortion coefficients
            }
        ]
    )

    # Launch the easy_handeye2 hand-eye calibration publisher
    handeye_node = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            PathJoinSubstitution([FindPackageShare("easy_handeye2"), "launch",
"publish.launch.py"])
        ),
        launch_arguments={
            'name': 'my_handeye_calib2',
        }.items()
    )
```

With all these software components, our transform tree includes:

- The robot frames
- The camera frames
    - published by the camera node
    - camera → base transform published by the easy_handeye2 calibration script
- The ArUco marker frame
    - published by the OpenCV publisher node

7.3 Calibration

To use the setup described above, we need to calibrate the camera to match the camera visual space and the robot space in a common frame of reference. We use the

easy_handeye2 library to compute the transform between the camera and the robot base (eye-on-base).

Steps:

1. Enable FCI and MoveIt (e.g. the moveit.launch.py launch file).
2. Move the robot using a MoveIt RViz interface to sample diverse poses of the end-effector.
3. Attach the marker to the end-effector of the robot, using a 3D printed holder. It doesn't matter how the marker is placed on the end-effector.
4. Start the required nodes:
    a. realsense2_camera node (publishes camera frames)
    b. Marker tracking node (publishes camera_link → aruco_marker).
    c. robot_state_publisher + /tf from the robot, through the MoveIt launch file for example
    d. (Optional) Run rqt to visualize the camera feed and make sure the marker is in the frame
5. Run the calibration GUI:
   2 options:
    a. Run the calibrate.launch.py file with the necessary launch arguments:

```
ros2 launch easy_handeye2 calibrate.launch.py calibration_type:=eye_on_base
name:=my_handeye_calib, robot_base_frame:=base
robot_effector_frame:=fr3_link8 tracking_base_frame:=camera_link
tracking_marker_frame:=aruco_marker_frame freehand_robot_movement:=true
```

    b. Create a new launch file that uses the calibrate.launch.py file, for example handeye_calibrate.launch.py:

```
from launch import LaunchDescription
from launch.actions import IncludeLaunchDescription
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch_ros.substitutions import FindPackageShare
from launch.substitutions import PathJoinSubstitution

def generate_launch_description():
    easy_handeye_calibrate = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            PathJoinSubstitution([FindPackageShare("easy_handeye2"),
"launch", "calibrate.launch.py"])
        ),
        launch_arguments={
            "calibration_type": "eye_on_base",
            "name": "my_handeye_calib3",
            "robot_base_frame": "base",
            "robot_effector_frame": "fr3_link8",
            "tracking_base_frame": "camera_link",            # static
RealSense base frame
            "tracking_marker_frame": "aruco_marker_frame",   # published by
ArUco/AprilTag node
            "freehand_robot_movement": "true"
        }.items()
    )

    return LaunchDescription([easy_handeye_calibrate])
```

i. Change the launch arguments as needed, especially the calibration type (‘eye_on_base’ or ‘eye_in_hand’), the name of the calibration and the name of the frames
ii. run the launch file:
```
ros2 launch easy_handeye2 handeye_calibrate.launch.py
```

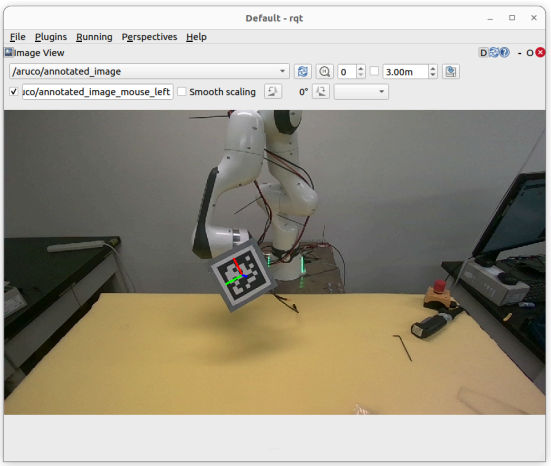
6. Collect samples in the GUI while moving the robot and ensure a diverse set of viewpoints.Save the calibration result. The tool will store a static transform: camera_link → base to the .yaml config file (in **~/.ros2/easy_handeye2/calibrations**)

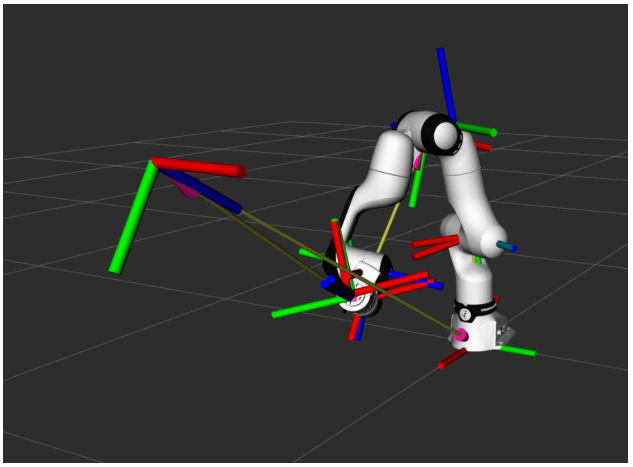
7.4 Use

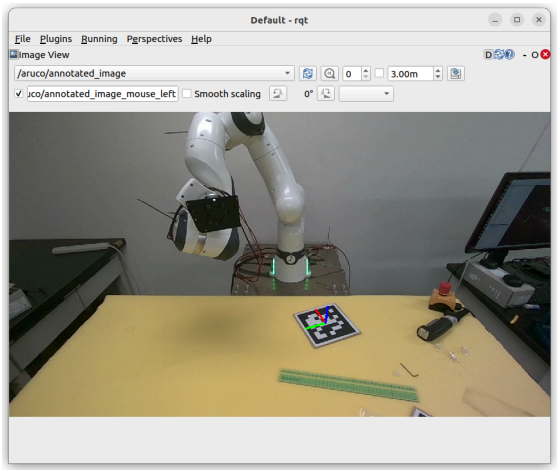
Once the camera is calibrated and publishes its transform relative to the robot, you can use it in the teleoperation or planning pipeline.

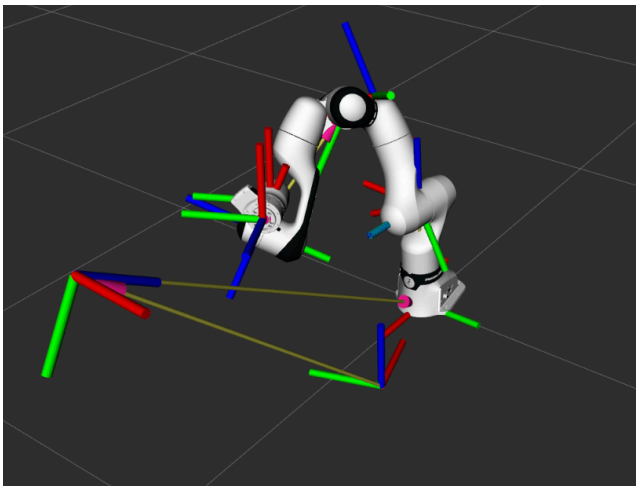
Example Use Cases:

- Locating a target with an ArUco marker
  - Place the marker on or near the object of interest (e.g., nozzle for insertion).
  - The marker’s TF gives a reliable reference frame in robot coordinates.
  - You can extract this TF and use it as a target for MoveIt planning or Servo input.
- Using raw image and depth data
  - Topics /camera/color/image_raw and /camera/depth/image_rect_raw can be consumed in real time.
- Future integration
  A model-based or learning-based vision module can be added to:
  - Segment objects,
  - Estimate deformation or grasp affordances,
  - Adapt robot commands accordingly.






Default - rqt
File Plugins Running Perspectives Help
Image View
/aruco/annotated_image
0
3.00m
/aruco/annotated_image_mouse_left
Smooth scaling
0°



8. Tactile feedback

8.1 Overview

Tactile sensing provides direct contact information that complements vision, which is especially useful when handling deformable objects like soft tubes. While visual feedback helps locate the object in space, tactile feedback gives insight into contact force, slip, and surface deformation, enabling more adaptive, compliant control.
In this setup, we integrate a 9DTact sensor, a compact, open-source vision-based tactile sensor, capable of reconstructing surface shape and estimating 6D contact force.

8.2 Setup

8.2.1 Hardware

The 9DTact sensor is composed of:

- A silicone gel surface
- An internal LED panel for illumination, connected via USB
- A USB camera that observes surface deformation

To build the 9DTact sensor from scratch, follow the video fabrication procedure.

Attach the sensor securely to the LEAP Hand or an experimental test rig using a 3D-printed frame. Ensure the contact surface is facing the target object (e.g., tube).

8.2.2 Software and calibration

Follow the official installation procedure to install and calibrate the sensor.
If needed, initialize conda for your current terminal session by running:

```
/home/user/miniconda3/bin/conda init
```

Notes for calibration:

Camera Calibration

Place a calibration board ~15 mm from the gel surface. Adjust camera focus manually.

Run:

```
cd shape_reconstruction
conda activate 9dtact
python _1_Camera_Calibration.py
```

If OpenCV fails to detect corners, tune the filtering thresholds in the script, especially:

```
        diff[diff < 5] = 0  # change this threshold for your sensor
```

If

Sensor Calibration

Use a 4.0 mm ball to press the surface.

Run:
`python _2_Sensor_Calibration.py`
This step generates reference deformation profiles used for shape and force reconstruction.

8.2.3 ROS2 integration

For ROS2 integration, we will modify the ROS module to port it to ROS2.
The ROS-compatible scripts are in the shape-force_ros/ folder.

In the GitHub repository for this setup, we add a 9dtact package that ports the shape-force_ros package to ROS2. It includes:

- shape_reconstruction folder for access to the camera and sensor calibration
- sensor_node.py: port of _1_Sensor_ros.py
    - publishes the deformation representation image to /deformation_representation
    - opens a window for visualization
- reconstruction_node.py: port of _2_Shape_Reconstruction_ros.py
    - Opens a 3D visualization of the estimated deformed surface of the sensor
    - needs the reference image from sensor_node
- (not ported to ROS2) _3_Force_Estimation_ros.py
- (not ported to ROS2) _4_Shape_Force_ros.py

Try the nodes with:
```
ros2 run 9dtact sensor_node
ros2 run 9dtact calibration_node
```

Add a 9DTact node to the launch file:
```
# Launch the 9DTact sensor node
sensor_node = Node(
    package='9dtact',
    executable='sensor_node',
    name='sensor_node',
    output='screen',
)
```

Note: the package has been renamed tact9d to respect python packages naming conventions.

8.3 Use case: Haptic feedback

We can use the deformation representation image as feedback data in the ROS2 pipeline, directly from the published ROS2 topic, or save the frames from the 9DTact sensor pipeline to avoid transmitting heavy data through ROS2 topics.
Additionally, we can use the data from the sensor to provide haptic feedback of what the robot is touching. For this usage, the shape reconstruction logic can be used to provide feedback according to the estimated height of the deformation, for example, max height, mean height or a metric designed to indicate the quality of the grasp on an object.

We design a Multi-Tactile driver to add to the teleoperation device, as described in this guide. This driver consists of:

- ESP32-C6 development board (e.g., ESP32-C6-DevKitC)
- USB-C or USB-A data cable (not power-only)
- PWM vibrotactile motors mounted on finger attachments

The python script uploaded to the dev board is main.py, also available in the package fr3_leap_recorder.

9. Data collection

To collect datasets of human demonstrations for model training, we develop a pipeline to record the important data to a file.
The collected datasets are structured as `frame_000/`, `frame_001/`, … with per-frame images like `color_image1.jpg`, `depth_image1.png`, etc.

For this purpose, the script fr3_leap_recorder.py in the fr3_leap_recorder package collects the following data for each frame:

- camera RGB data (2 cameras)
- camera depth data (2 cameras)
- 9DTact sensor data, raw image (3 sensors)
- Joint angles data, text file (7 arm joints + 16 hand joints)

10. Troubleshooting and Debugging Tools

There are many interconnected components in this system so issues are to be expected. This section compiles a few issues I've encountered with the Franka FR3, libfranka, MoveIt Servo, the LEAP Hand, or DexCap, along with diagnostic tools that will help you debug the system effectively.

10.1 libfranka exceptions

Visit the Franka Control Interface documentation page for a description of the errors.
The full list of errors that can be encountered when using libfranka is available in the FCI API guide.

10.1.1 Communication constraints violation

Franka troubleshooting page for more details):
Problem: libfranka::Exception: communication_constraints_violation
This exception indicates that the robot didn't receive control commands within the required real-time deadline. Causes include:

- Packet loss
- High round-trip latency
- CPU scheduling issues on your Linux PC

Solution Checklist:

1. Direct Ethernet connection
    - Always connect the control box (not the arm) directly to the Linux PC.
    - Avoid using an unmanaged Ethernet switch or router in between.
2. Ping Test (10 kHz for 10 seconds):
   Run this command:
   ```
   sudo ping 192.168.1.11 -i 0.001 -D -c 10000 -s 1200
   ```
   Output should show:
   0% packet loss
   max rtt < 1ms
   If the max round-trip time exceeds 1ms, communication may fail, as explained in the network requirements section.
3. Disable the CPU frequency scaling:
   Set your CPU governor to performance in the settings (cpufrequtils required).
4. Use a Real-Time Kernel
   For better determinism, install a PREEMPT-RT kernel, though it may conflict with NVIDIA drivers or GUI responsiveness.

10.1.2 Motion constraints violations

There are many conditions that the commands have to fulfill, including:

- Joint limits
- Velocity, acceleration and jerk limits

- Signal continuity
- Torque limits and power limit

A planner like MoveIt should already take these requirements into account. In pose tracking, make sure to have the PID tuned to be sufficiently smooth. When making a custom controller, rate limiters have to be included.
Read the Errors due to non-compliant commanded values and the Robot and interface specifications.

10.1.3 Behavioral errors

These errors are caused by collision or self-collision detection, or in some cases, an abrupt motion. Read the Behavioral errors.
To interact with the environment, we need to increase the torque thresholds.
Use a service call:

```
ros2 service call /service_server/set_force_torque_collision_behavior
franka_msgs/srv/SetForceTorqueCollisionBehavior
"{lower_torque_thresholds_nominal: [10,10,10,10,10,10,10],
upper_torque_thresholds_nominal: [12,12,12,12,12,12,12],
lower_force_thresholds_nominal: [50,50,50,50,50,50],
upper_force_thresholds_nominal: [60,60,60,60,60,60]}"
```

10.2 Parameter Errors in ROS 2

Problem: Launch fails with: parameter '<...>' is not set
This typically happens when:
The controller or node expects parameters that aren't declared or passed.
A config YAML file is missing or not loaded correctly.

Solution:
Check that all required parameters are declared in your YAML config.
There may be a version mismatch where the node expects a parameter undefined in the config files.

(Optional) Checkout to a stable version:
The following process is how I resolved a specific issue where the default MoveIt launch file failed due to missing parameters on real hardware, while working fine with dummy hardware.
Create the workspace and clone the repository:

```
mkdir -p ~/franka_ros2_ws/src
cd ~/franka_ros2_ws/src
git clone https://github.com/frankaemika/franka_ros2.git
```

Check out a stable version (e.g., v1.0.0):

```
cd franka_ros2
git fetch --tags
git checkout v1.0.0
```

(Optional fix) If the franka.repos file is missing (as in tag v1.0.0), download or recreate it manually in the src directory. This file is used to pull dependencies using vcs.
Install dependencies and build:

```
cd ~/franka_ros2_ws
```

```
rosdep install --from-paths src --ignore-src -r -y
colcon build --cmake-args -DCMAKE_BUILD_TYPE=Release
source install/setup.bash
```

Always source the workspace in every terminal.

10.3 Debugging the ROS 2 System

When things don't work as expected (robot doesn't move, hand freezes, pose not received), use the following tools for basic introspection:

- List active nodes:
  ```
  ros2 node list
  ```

- Check topic connections:
  ```
  ros2 topic list
  ros2 topic info /servo_node/pose_target
  ros2 topic echo /servo_node/status
  ```
  Use the --verbose option to display more information.

- Check frame transforms (TF tree):
  ```
  ros2 run tf2_tools view_frames
  evince frames.pdf
  ```

- Check controller status:
  ```
  ros2 control list_controllers
  ros2 control list_hardware_interfaces
  ```

- Check the ROS_DOMAIN_ID:
  ```
  echo $ROS_DOMAIN_ID
  ```

  Terminals with the same **ROS_DOMAIN_ID** can interact with each other. Make sure your terminals have the same domain ID, otherwise it prevents discovery between terminals.

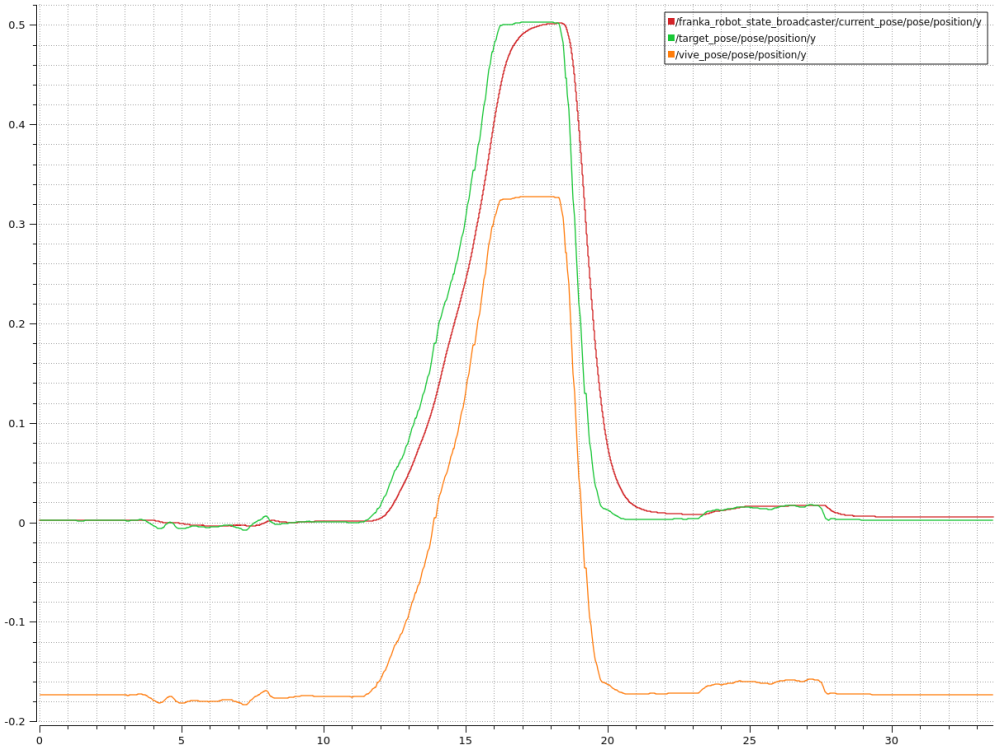
  Different workstations on the same network may interact with each other (and conflict!) if they have the same domain ID or if it's not set. If it's unset, you can set it to a number with:

  ```
  export ROS_DOMAIN_ID=0
  ```

10.4 Visualization and Analysis Tools

10.4.1 PlotJuggler

Use PlotJuggler for real-time plotting of ROS 2 topic data.



Install via:

```
sudo apt install ros-humble-plotjuggler ros-humble-plotjuggler-ros
```

To start:

```
ros2 run plotjuggler plotjuggler
```

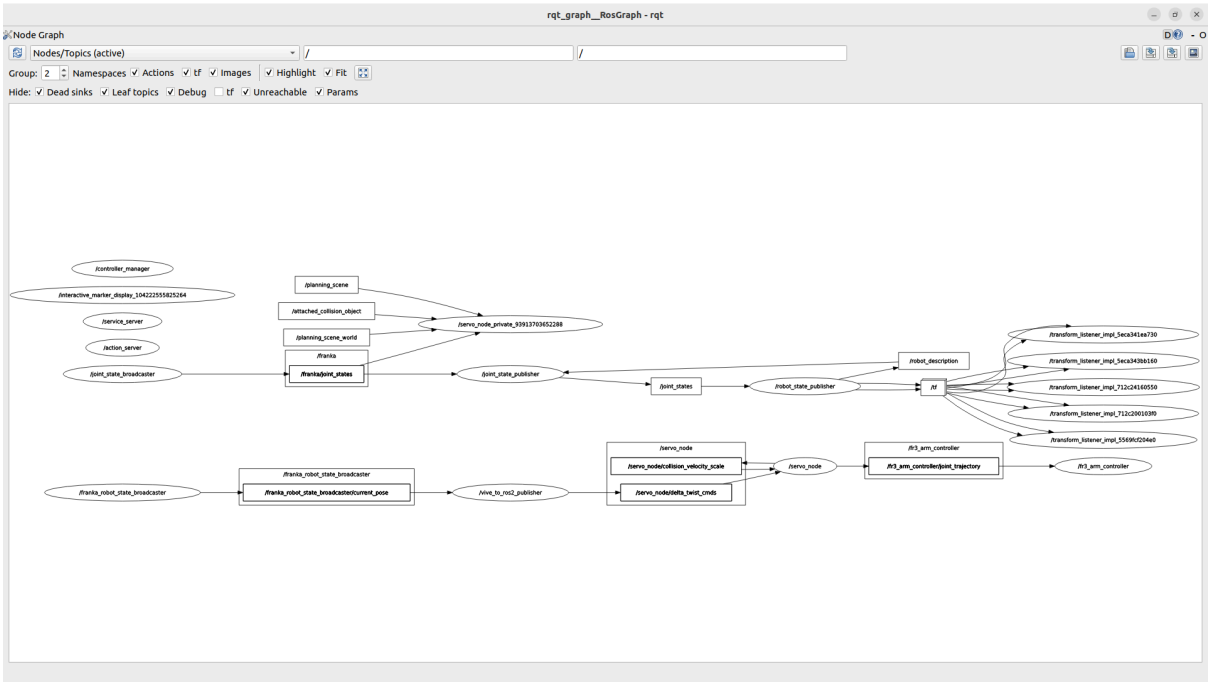
Then click Connect to ROS 2 and add topics like:

- /servo_node/delta_twist_commands
- /joint_states
- /franka_robot_state_broadcaster/current_position
- /target_pose

10.4.2 rqt_graph

Visualize the node/topic architecture:

```
rqt_graph
```

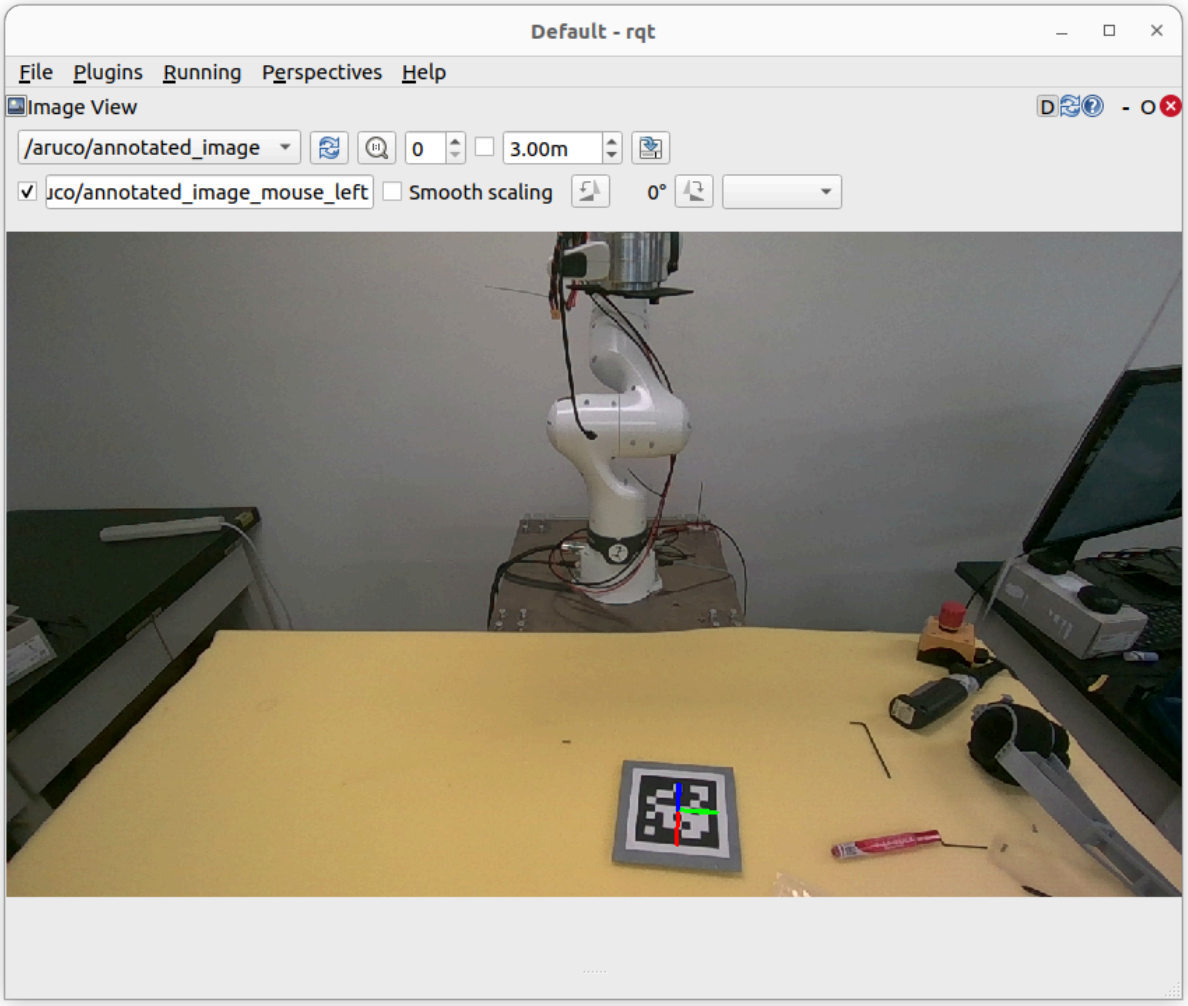


Useful to confirm whether:

- Your UDP receiver to ROS2 publisher is publishing to /target_pose
- Servo node is actually subscribing
- All the nodes are correctly launched and connected to each other via the topics

10.4.3 rqt

Use the command ‘rqt’ in a terminal to open the rqt window. rqt allows you to run various plugins, including a visualization of the ROS2 topics with image type messages:



10.5 Vive Tracker Issues

Problem: Tracker shows "Lost Tracking"
This usually means the tracker is outside the mapped space or the cameras of the tracker have been occluded.

Fix:
Place the tracker in the center of the mapped room.
Re-run SteamVR room setup if you changed tracker layout or position

10.6 MediaPipe or LEAP Hand Doesn’t Respond

Problem: Hand doesn’t move when retargeting script runs
Checklist:

- Confirm USB and power connection to the LEAP Hand
- Confirm that the Python control script is publishing commands (add print statements or logging)
- Make sure --retargeting-type is correctly set (position or velocity)
- Check the connection of the motors in the Dynamixel Wizard.

10.7 MoveIt Servo Doesn’t Move Robot

Checklist:

- Confirm you're publishing to the correct input topic:
  - /target_pose for pose tracking
  - /servo_node/delta_twist_cmds for twist control
- The message types of the ROS2 topics are correct
- The frame of the commands is correct
  - In pose tracking, the robot_link_command_frame must be the end-effector frame
  - In servoing, it must be the base frame
- Open RViz and check robot state matches reality
- Run a keyboard test to isolate the problem

10.8 9DTact Calibration problems

conda activate 9dtact

conda: command not found

If you get the following error when running:

```
conda activate 9dtact
conda: command not found
```

Run:

```
eval "$(/home/user/miniconda3/bin/conda shell.bash hook)" # Change to your conda install path
conda activate 9dtact
```

ModuleNotFoundError: No module named 'shape_reconstruction'

Run from the shape_reconstruction folder while pointing at the parent folder:

```
(9dtact) user@user-Z790-Steel-Legend-WiFi:~/9DTact$ cd ~/9DTact/shape_reconstruction
PYTHONPATH=.. python _1_Camera_Calibration.py
```

Camera not detected

If you see `can't open camera by index` or `Camera index out of range`, check the channel where your sensor is connected:

```
v4l2-ctl --list-devices
```

In /9DTact/shape_reconstruction/shape_config.yaml, edit the variable camera_channel to match the channel of the 9DTact USB camera.

10.9 Connected devices

10.9.1 Realsense cameras

rs-enumerate-devices

Sometimes the camera doesn't start unless you unplug and plug it back.

10.9.2 Other USB cameras

```
v4l2-ctl --list-devices
```

10.9.2 All USB devices

`lsusb`
or `lsusb -t` to display the tree

10.9.3 USB Hub / RealSense Startup Delays

Problem:
pyrealsense2 initialization (rs.context()) is extremely slow (30–50 s) when certain USB devices or hubs are connected. The cameras eventually work, but startup is delayed and error messages may appear in dmesg such as:

```
usb usb2-port3: Cannot enable. Maybe the USB cable is bad?
usb 2-9-port4: config error
usb 2-9.4: device not accepting address 23, error -71
```

This can happen even if the RealSense cameras are not physically connected to the problematic hub.

Checklist / Steps to Fix:

1. Identify problematic USB hubs
   Run:
   ```
   dmesg -w | grep usb
   lsusb -t
   ```

   Look for repeating "Cannot enable" or "config error" messages.

2. Connect RealSense cameras to dedicated USB 3 ports
   Avoid hubs or daisy-chaining for RealSense devices.
   Check with lsusb -t that the cameras are on a 5 Gb/s (USB 3) or higher bus.

3. Keep low-bandwidth devices on hubs
   Connect HID devices, LED boards, or platform cameras to hubs.
   Prefer powered hubs for multiple devices to avoid undervoltage.
4. Avoid shared root hubs between RealSense and other USB 2 devices

USB 2 cameras or devices sharing the same root hub as a RealSense can delay enumeration.

5. Test startup

```
python3 - <<'EOF'
import time, pyrealsense2 as rs
t0 = time.time()
ctx = rs.context()
print("Context init:", time.time()-t0, "s")
print("Devices:", [d.get_info(rs.camera_info.serial_number)
for d in ctx.devices])
EOF
```

   Startup should now be ~0.5–1 s.
   If slow, revisit hub placement and port selection.

6. Unplug / replug devices if needed
   Some hubs or devices may still need a physical reconnect to initialize properly.